Intermediate — a Flask app with a form — Solution

> Back to the exercise · Hint 1 · Hint 2

Story: the real web-app pattern Streamlit hides — you write the route and the HTML yourself. **If not:** you never learn what a route or a template actually is, and you can't build anything Streamlit doesn't already shape for you.

The Flask app

```
# practice/app_flask.py   —   run:  flask --app app_flask run
from flask import Flask, request, render_template_string, jsonify

from pipeline import classify

app = Flask(__name__)

PAGE = """
<!doctype html>
<title>Sentiment</title>
<form method="post">
  <input name="text" value="{{ text }}" placeholder="Type something">
  <button>Classify</button>
</form>
{% if label %}<p>Result: <b>{{ label }}</b></p>{% endif %}
"""

@app.route("/", methods=["GET", "POST"])
def home():
    label = None
    text = ""
    if request.method == "POST":
        text = request.form.get("text", "")
        if text.strip():
            label = classify(text)
    return render_template_string(PAGE, label=label, text=text)

@app.post("/classify")
```

```
def classify_api():
    data = request.get_json(silent=True) or {}
    text = data.get("text", "")
    if not text.strip():
        return jsonify({"error": "empty text"}), 400
    return jsonify({"label": classify(text)})
```

How it works

- **The `/` route** handles both `GET` (show an empty form) and `POST` (the form was submitted). On POST you read `request.form["text"]`, call the pipeline, and re-render the **same** page with the result filled in.
- **The template** is HTML with Jinja `{{ slots }}` — the fixed page written once, the changing parts (`text`, `label`) passed in by `render_template_string`. In a real app this lives in a `templates/` file and you use `render_template("page.html", ...)`.
- **The `/classify` route** is the JSON version — the same thing an API client (or a JavaScript frontend) would call. It validates the input and returns `400` on empty text instead of crashing.

Testing it without a server

Flask's `test_client()` exercises the routes in-process — no running server, like FastAPI's `TestClient`:

```
# practice/test_flask.py   —   python -m practice.test_flask
from app_flask import app

def test_form_and_api() -> None:
    client = app.test_client()
    get_ok = client.get("/").status_code == 200
    page = client.post("/", data={"text": "I love it"}).get_data(as_text=True)
    api = client.post("/classify", json={"text": "terrible"}).get_json()
    empty = client.post("/classify", json={"text": ""}).status_code
    result = "OK"
    if not get_ok or "positive" not in page:
        result = "WRONG"
    if api != {"label": "negative"} or empty != 400:
        result = "WRONG"
    print(f"case 1 (form + api + empty) -> {result}")

if __name__ == "__main__":
    test_form_and_api()
```

Expected: `GET /` returns the form (200); submitting "I love it" re-renders with "positive"; `POST /classify {"text": "terrible"}` returns `{"label": "negative"}`; empty text returns `400`.

Why Flask over Streamlit: you control the HTML, the routes, and the response — a real, extensible web app you can style, add pages to, and put behind `gunicorn`. The cost is the handful of lines Streamlit wrote for you.